

Homework 5: GPU vs. CPU SIMD

Due: 10:00 P.M. (ET) on Monday, November 10th, 2025

Last Updated: October 29, 2025

In this homework you will experiment with parallelizing programs with CPU SIMD and GPU. You will learn how to write basic GPU code and utilize GPU resources to execute your program in parallel, and practice CPU SIMD parallelism using AVX intrinsics.

Contents

1	Getting Started	1
2	Introduction	1
2.1	Goals for This Lab	1
3	Part 1: Scalar Mandelbrot	3
3.1	CPU: Reference Implementation (Already Written)	3
3.2	GPU: Your Implementation	4
4	Part 2: Vector-Parallel Mandelbrot	6
4.1	CPU: SIMD Programming Model + SIMD Execution	6
4.2	GPU: SPMD Programming Model + SIMD Execution	7
5	Turn-in	9

1 Getting Started

Getting the Starter Code

You can get the starter code by cloning the lab repository:

```
git clone https://github.com/spe-fall25/homework5_<username>.git
```

2 Introduction

2.1 Goals for This Lab

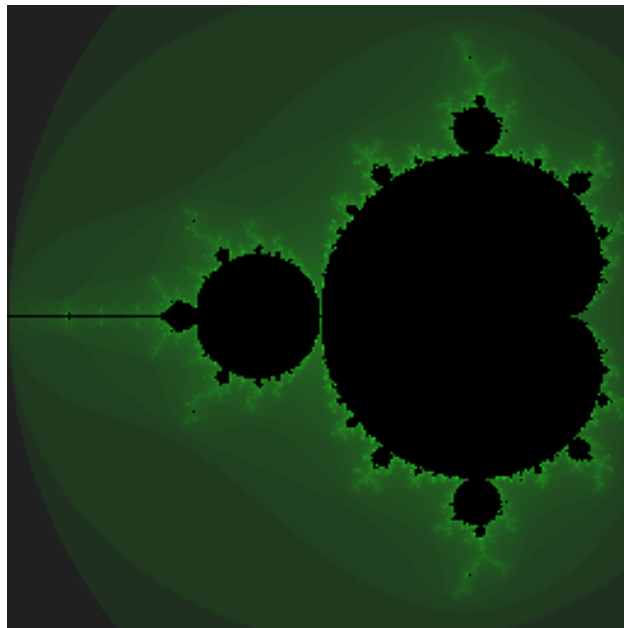
One of the most fundamental concepts which we'll be repeatedly returning to throughout this course is **parallelism**. Methods of exploiting parallelism can take a variety of different forms at

the hardware level, and can be exposed in a variety of different ways at the software level. In this lab, we'll be looking at:

1. The different fundamental **levels** of parallelism which exist in modern CPUs and GPUs: vector parallelism, instruction-level parallelism, multi-threaded parallelism, and multi-core parallelism.
2. How the different types of parallelism which exist on GPUs **correspond** to analogous (and perhaps more familiar) types of parallelism which exist on CPUs.

Although GPUs and CPUs can look quite different at first, we hope that this lab will help you understand these different hardware platforms in terms of their **deep similarities**. As we'll see, modern CPUs and GPUs have actually converged towards using many of the same architectural features for efficiently processing throughput-oriented workloads, although they choose to expose these features to the programmer in different ways.

To explore the various types of parallelism we're interested in, we'll be building up a series of different approaches to computing visualizations of the [Mandelbrot fractal](#) on CPUs and GPUs. That means you'll be writing C++ and CUDA code to generate pictures that look like this:



For this lab, we'll be looking at two ways of rendering Mandelbrot fractals:

1. **Sequential scalar** code, with no explicit parallelism at all.
2. **Vector-parallel** code, using a "Single Instruction, Multiple Data" (SIMD) execution model.

We'll look at what implementing each of these strategies looks like on both CPUs and GPUs. Along the way, we'll be teaching you a tiny bit of CUDA (you're not expected to have ever used CUDA before).

3 Part 1: Scalar Mandelbrot

3.1 CPU: Reference Implementation (Already Written)

In this lab, we'll be studying ways to parallelize the following computation:

```
void mandelbrot_cpu_scalar (
    uint32_t img_size,      /* usually greater than 100 */
    uint32_t max_iters,     /* usually greater than 100 */
    uint32_t *out           /* buffer of 'img_size * img_size' integers */
) {
    for (uint32_t i = 0; i < img_size; i++) {
        for (uint32_t j = 0; j < img_size; j++) {
            float cx = (float(j) / float(img_size)) * 2.5f - 2.0f;
            float cy = (float(i) / float(img_size)) * 2.5f - 1.25f;

            float x2 = 0.0f;
            float y2 = 0.0f;
            float w = 0.0f;
            uint32_t iters = 0;
            while (x2 + y2 <= 4.0f && iters < max_iters) {
                float x = x2 - y2 + cx;
                float y = w - x2 - y2 + cy;
                x2 = x * x;
                y2 = y * y;
                w = (x + y) * (x + y);
                ++iters;
            }
            out[i * img_size + j] = iters;
        }
    }
}
```

This code populates a two-dimensional buffer of integers which can be used to visualize the [Mandelbrot fractal](#), if the integers are converted to colors (for this reason, we'll call each individual element of the out buffer a "pixel"). You can try running this code with:

```
python3 execute.py mandelbrot_cpu.cpp
```

to measure its run time and see what kind of images it produces (checkout the .bmp files after you run the program).

There are many interesting things which can be said about the mathematics of the Mandelbrot fractal and algorithms for computing it, but for the purposes of this lab, we're not concerned with where this algorithm came from or how to intuitively interpret what it's doing.

Instead, we're concerned with the properties of this Mandelbrot workload from a **performance engineering** perspective; we just want to make it run *fast*. From that perspective, some of the most important properties of this workload are:

1. **Every pixel is independent:**

The work done at each pixel location i, j doesn't depend on the work done for any other values of i or j .

2. **There are a lot of pixels:**

For typical image sizes, we'll need to compute tens of thousands of different pixels.

3. **Computation dominates:**

Because we only need to access main memory once per pixel (to write to the out buffer), we can expect most of the run time to be spent doing math in the inner loop.

4. **Different pixels need different amounts of work:**

The inner while loop terminates after an unpredictable number of iterations, which might be different for different pixels.

The sample code above implements this algorithm entirely sequentially, processing one pixel at a time. However, from properties 1 and 2, we can see that there's a lot of potential for exploiting parallelism in this workload by processing multiple pixels simultaneously. (In fact, problems like this are sometimes called "[embarrassingly parallel](#).")

In Part 2, we'll be looking at one way to introduce parallelism into this program, using vector instructions. But first, we'll dip a toe into GPU programming by writing a CUDA version of this *sequential* implementation.

3.2 GPU: Your Implementation

As a first exercise in CUDA, we'll write a *direct translation* of the sequential CPU algorithm given above for generating Mandelbrot fractals. Even though this version will run on the GPU, it will use no parallelism at all.

For context, we'll first note a few basic facts about CUDA:

1. **CUDA is a dialect of C++** that supports running code on the GPU. Most features of C++ are supported in CUDA GPU code, and use the same syntax as they do in regular C++ on your CPU. ([Reference](#))

2. **CUDA is written in .cu files**, which can contain a *mix* of code which runs on the GPU and code which runs on the CPU. The parts of the code in a .cu file which run on the CPU are written in ordinary C++, plus a few extensions that let it interact with the GPU. ([Reference](#))
3. **Pointers** in CUDA can point to **either CPU or GPU** memory. You can't know solely based on a pointer variable's type whether it points to CPU or GPU memory; you also need to know where the pointer came from. It is safe for CPU code to *hold* pointers to GPU memory; but if CPU code tries to *load* or *store* through a pointer to GPU memory, or vice-versa, it will crash the program. ([Reference](#))
4. The **__global__** keyword marks a function as running **on the GPU**. Specifically, **__global__** declares a "CUDA kernel," which is a function which runs on the GPU but which can be "launched" from the CPU. ([Reference](#))
5. **CPU code can launch CUDA kernels** using the special syntax

```
kernel_name<<<N, M>>>(arg_1, ..., arg_n)
```

where N and M are integers which control parallelism, kernel_name is a function declared with the **__global__** keyword, and arg_1, ..., arg_n are arguments passed to the kernel. ([Reference](#))

In mandelbrot_gpu.cu, you'll first find two functions to be implemented:

```
__global__ void mandelbrot_gpu_scalar(
    uint32_t img_size,
    uint32_t max_iters,
    uint32_t *out /* pointer to GPU memory */
) {
    /* your (GPU) code here... */
}

void launch_mandelbrot_gpu_scalar(
    uint32_t img_size,
    uint32_t max_iters,
    uint32_t *out /* pointer to GPU memory */
) {
    /* your (CPU) code here... */
}
```

Because the function mandelbrot_gpu_scalar is marked **__global__**, the CUDA compiler will compile it to run on the GPU.

The testing + benchmark harness provided with the lab code will not launch mandelbrot_gpu_scalar directly; instead, the provided lab code will call the CPU function launch_mandelbrot_gpu_scalar,

passing it a pointer to a GPU memory buffer to be populated by a CUDA kernel. It is then the responsibility of the `launch_mandelbrot_gpu_scalar` function to actually launch the kernel that will fill that buffer.

To implement the scalar Mandelbrot CUDA kernel, you can follow these steps:

1. Copy the code from `mandelbrot_cpu_scalar` into the body of `mandelbrot_gpu_scalar`
2. Add the line

```
mandelbrot_gpu_scalar<<<1, 1>>>(img_size, max_iters, out);
```

in the body of `launch_mandelbrot_gpu_scalar`. This will launch the kernel with no parallelism; it will run on the GPU as if it were a single-threaded CPU program.

If this seems straightforward, that's because it is! This step in the lab is only meant to start getting you familiar with some of the basic elements of CUDA.

Once you've filled in the scalar CUDA implementation, you can run it with:

```
python3 execute.py mandelbrot_gpu.cu
```

Question 1 for the write-up: How does the run time of the scalar GPU implementation compare to the scalar CPU implementation? What factors do you think might contribute to each of their run times?

4 Part 2: Vector-Parallel Mandelbrot

Now it's time to start exploiting some of the parallelism available in the Mandelbrot workload. There are many different types of parallel execution exposed in both CPUs and GPUs that we might try to apply to this problem, but for this lab we'll be focusing on *vector parallelism*, also known as "Single Instruction, Multiple Data" (SIMD) parallelism. We'll start with a vectorized CPU implementation, and then see how the same principles of SIMD execution also show up in GPUs (albeit exposed to programmers in a very different way).

4.1 CPU: SIMD Programming Model + SIMD Execution

For our vectorized CPU implementation, we'll be targeting x86 with AVX extensions. For 32-bit integer and float types, the vectors provided by AVX are $256 / 32 = 8$ elements wide. As a result, the strategy we recommend for vectorizing the Mandelbrot workload is to generate one contiguous 8-wide row of pixels at a time.

Your task is to implement the `mandelbrot_cpu_vector` function in `mandelbrot_cpu.cpp` using explicit vector operations through the [AVX intrinsics](#) (check under AVX family for instructions on 256 bit vectors). You can assume that the `img_width` is always evenly divisible by 8.

Things you might want to consider include:

- Think about what needs to happen to the structure of the outer loops.
- Think about how you're going to fill vectors with the right c_x and c_y values for a given row of 8 pixels.
- Think about how you're going to handle control flow. How do you deal with the fact that different pixels have different numbers of inner loop iterations?

Some very useful intrinsics you might want to use for this program are:

Intrinsic	Meaning
<code>_mm256_set1_ps(float a)</code>	Sets all 8 words of a vector to the same fp value a
<code>_mm256_set1_epi32(int a)</code>	Sets all 8 words of a vector to the same int value a
<code>_mm256_set_ps(e7, e6, ..., e0)</code>	Set 8 words of a vector with 8 values <i>in backwards order</i>
<code>_mm256_add_ps(...)</code>	Add two fp vectors element-wise
<code>_mm256_sub_ps(...)</code>	Subtract fp vectors element-wise
<code>_mm256_mul_ps(...)</code>	Multiply fp two vectors element-wise
<code>_mm256_cmp_ps(...)</code>	Generate mask based on the element-wise comparison of two fp vectors
<code>_mm256_blendv_ps(...)</code>	Blend two vectors element-wise according to the mask
<code>_mm256_storeu_si256(...)</code>	Write all 8 words of a vector contiguously into memory at a given address

You can test your `mandelbrot_cpu_vector` code by running `mandelbrot_cpu.cpp`:

```
python3 execute.py mandelbrot_cpu.cpp
```

Question 2 for final write-up: How did you initialize c_x and c_y ? How did you handle differences in the number of inner loop iterations between pixels in the same vector? How does the performance of your vectorized implementation compare to the scalar versions?

4.2 GPU: SPMD Programming Model + SIMD Execution

What we just did was painful. The CPU scalar version of Mandelbrot was much easier to read and write than the vector version.

CUDA is designed to let you write code which *looks* like the scalar version, but which *executes* on SIMD hardware like the vector version, with the performance advantage that entails.

Earlier, when we wrote the GPU scalar version of Mandelbrot, we set both of the launch parameters which control the parallelism of the `mandelbrot_gpu_scalar` kernel to 1:

```
mandelbrot_gpu_scalar<<<1, 1>>>(img_size, max_iters, out);
```

Setting those parameters to 1, 1 caused the kernel to run without any parallelism. By setting those parameters to higher values, we can unlock several different kinds of parallelism. We'll cover all of them over the next two weeks, but for now we'll show how to set them, and update our kernel, to exploit SIMD parallelism.

We can think of these parameters as specifying the size of an iteration space over which the kernel will be instantiated. This is often called a “Single **Program** Multiple Data” (SPMD) programming model.

NVIDIA's GPU hardware natively executes all instructions on 32-wide vectors, and if we set the inner dimension of the iteration space to a value ≤ 32 , all instances of our kernel will be running in different lanes of a single SIMD vector. In fact, even our GPU “scalar” version was secretly executing on just one lane of a 32-wide vector!

Our final goal for this lab is to implement a vector-parallel Mandelbrot on the GPU, directly analogous to the vector-parallel version we wrote for CPU. To do this, we need to implement two functions:

```
__global__ void mandelbrot_gpu_vector(
    uint32_t img_size,
    uint32_t max_iters,
    uint32_t *out /* pointer to GPU memory */
) {
    /* your (GPU) code here... */
}

void launch_mandelbrot_gpu_vector(
    uint32_t img_size,
    uint32_t max_iters,
    uint32_t *out /* pointer to GPU memory */
) {
    /* your (CPU) code here... */
}
```

As before, `launch_mandelbrot_gpu_vector` should launch `mandelbrot_gpu_vector`, but this time using the dimensions `<<<1, 32>>>`.

When we do that, the code inside `mandelbrot_gpu_vector` will run not once, but 32 times in parallel. From the perspective of the hardware, each of the 32 copies is executed on a different lane of a single SIMD vector. But if each copy is running the same code, how do they ever compute different values? In the same way that for the CPU vector implementation you had to initialize different lanes of your vectors with different data in order to get them to generate different pixels, we need a way for each instance of the kernel to know what pixel it's supposed to compute.

For our launch configuration, CUDA exposes the index of each kernel instance in a special integer variable called `threadIdx.x`. In the first lane of the implicit SIMD vector this will have the value 0, in the second it will have value 1, and so on.

Using that, your next job is to implement `mandelbrot_gpu_vector` so that it computes 32-wide rows of pixels at a time when launched by `launch_mandelbrot_gpu_vector`. Don't worry if the control flow is different across different instances of your kernel. Even though it executes in SIMD style, the hardware knows how to ensure that your program executes *as if* every instance were running independently.

As before, you can run your GPU code with Telerun:

```
python3 execute.py mandelbrot_gpu.cu
```

Question 3 for final write-up: How does the performance of your vectorized implementation compare to the scalar versions? Given how you implemented the vector-parallel CPU version with explicit SIMD, how do you think the GPU executes multiple kernel instances that run different numbers of iterations?

5 Turn-in

When you've written up answers to all of the above questions, turn in your write-up by uploading it to Gradescope, and commit and push your code to your Git repository.